

ASTRA

(AUTONOMOUS SMART TECHNOLOGY FOR ROBOTIC ASSISTANCE)

A Multi-Modal Robotic Platform with Remote, Voice, and Gesture Control,
Obstacle Safety, Live Streaming, and Human–Robot Interaction

ETIGOWNI HEMANTHESWAR GOUD	(R220419)
MANDADI SRIKEERTHI	(R220901)
HAJI AYESHA KOUSER	(R220493)
BALAREDDY GARI MANASA	(R220174)
DUDEKULA KULLAI SHAREEF	(R220840)
BOYA SRAVANI	(R220674)

Departments of Electronics & Communication Engineering and Computer Science & Engineering
Rajiv Gandhi University of Knowledge Technologies (RGUKT), RK Valley, Andhra Pradesh, India

ABSTRACT

ASTRA (Autonomous Smart Technology for Robotic Assistance) is a multimodal robotic platform that we built to explore how a single robot can be controlled in multiple ways through a unified embedded system. The platform supports three modes of control — remote control via the Blynk IoT app, voice commands processed through a Python pipeline, and hand-gesture recognition using OpenCV and MediaPipe — all running on an ESP8266 NodeMCU as the main controller. A shared obstacle detection routine using an HC-SR04 ultrasonic sensor enforces three-tier movement restrictions across all modes without any mode-specific code duplication. The system uses a distributed hardware design: an Arduino Uno handles TFT LCD eye animations for human-robot interaction, a Raspberry Pi Pico 2W handles live video streaming independently, and a Raspberry Pi Zero 2W runs a MobileNet-SSD object detection pipeline that identifies objects from the camera feed and announces them verbally in real time using text-to-speech. A dual buck-converter power architecture was critical to solving early stability issues caused by motor-induced voltage transients. This paper documents the actual implementation, the engineering problems we ran into, and how we fixed them.

Keywords — ESP8266, IoT Robotics, Blynk, UDP Communication, OpenCV, MediaPipe, Gesture Control, Speech Recognition, Obstacle Avoidance, Human–Robot Interaction, Distributed Embedded System, Power Management.

I. INTRODUCTION

Most robotic platforms built in academic settings are single-mode — they either respond to a mobile app or a voice command, rarely both. The practical problem with this is that a single interface breaks when conditions change: Bluetooth becomes unreliable in noisy RF environments, voice recognition degrades with background noise, and physical remote controls require line-of-sight. We wanted to build something more resilient — a robot that could switch between control methods depending on what the situation demanded.

ASTRA grew out of that idea. It is not trying to be a research platform in the conventional sense. It is an honest attempt at integrating three control modalities — remote, voice, gesture — into one embedded system, while also solving the real practical problems that appear during integration: power instability, conflicting communication APIs, command safety under broken connections, and making the robot feel interactive through a simple eye animation system. Beyond control and interaction, the platform also incorporates a Raspberry Pi Zero 2W running a MobileNet-SSD object detection pipeline that identifies objects from the live camera feed and announces them verbally in real time — a step toward giving the robot genuine awareness of its environment.

The robot defaults to Remote mode at startup. From there, saying “hello” activates the voice and gesture pipeline, and the user can switch modes through spoken commands. A unified obstacle safety routine — embedded in the ESP8266 firmware and shared across all modes — ensures the robot does not collide regardless of how it is being controlled. A companion Raspberry Pi Zero 2W module handles real-time object detection using MobileNet-SSD, detecting and verbally announcing objects in the camera’s field of view. The rest of this paper documents how each part works, what went wrong during development, and what we would change next time.

II. MOTIVATION

We started this project because we felt most low-cost robot builds available online were too limited. They did one thing — drive around using a phone app — and stopped there. There was no safety logic, no interaction, and no fallback when the connection dropped.

We also wanted to understand what it actually takes to integrate hardware and software together in a real system. Writing embedded C and Python separately is one thing; making them communicate reliably over a network while a robot is moving around on a noisy power supply is a completely different challenge. ASTRA was our attempt to work through those challenges in a structured way.

One more practical motivation: none of us had experience with distributed embedded systems before this. Distributing tasks across ESP8266, Arduino Uno, and Raspberry Pi Pico 2W — each handling a different aspect of the robot — gave us hands-on understanding of why professional robotics platforms are designed that way.

III. PROBLEM STATEMENT

Before we started designing ASTRA, we identified several concrete problems with existing low-cost robotic platforms:

- Bluetooth communication caps operational range at roughly 10 meters and is unreliable through walls. Internet-based communication removes this ceiling.
- Single-mode control has no fallback. If speech recognition fails or the camera is obstructed, the robot becomes uncontrollable.
- No obstacle logic means collisions depend entirely on operator reflexes. This is not acceptable for any real deployment.
- Shared power rails between motors and microcontrollers cause voltage drops during motor load events, resetting controllers and corrupting sensor readings. This is a real hardware problem that needs a real hardware fix.
- Command starvation: if the last movement command is not explicitly cancelled and the communication link drops, the robot keeps moving. A timeout is needed.
- Running computer vision on the microcontroller itself is not feasible. A laptop or more powerful processor is required for the gesture and voice pipelines.

ASTRA addresses each of these in the sections that follow.

IV. OBJECTIVES

1. Build a robot that supports Remote, Voice, and Gesture control in one unified firmware.
2. Replace Bluetooth with UDP over Wi-Fi to remove range limitations.
3. Implement a three-zone obstacle safety model enforced at the firmware level, active in all modes.
4. Solve power instability through a regulated dual buck-converter architecture.
5. Use a distributed controller design to keep each subsystem independent and maintainable.
6. Add a command timeout mechanism to prevent indefinite execution when communication breaks.
7. Build a simple HRI eye animation system that reflects the robot's movement state.
8. Create a foundation that can be extended with AI and autonomous navigation in future work.

V. LITERATURE SURVEY

A. IoT-Based Remote Robotic Control

Blynk and similar IoT platforms have enabled mobile-app-based robotic control over Wi-Fi and internet connections. These systems work well for remote control but do not integrate voice or gesture interfaces. Reported command latency through the Blynk cloud is typically 200–400 ms over a local network, which is acceptable for a slowly moving robot.

B. Vision-Based Gesture Recognition

MediaPipe Hands provides real-time 21-keypoint hand landmark detection that runs comfortably on a laptop CPU. Prior implementations for robotic control have shown that raw per-frame finger counting is unstable

without smoothing. Most published implementations do not describe a formal smoothing strategy — we implemented a majority-vote buffer specifically to address this.

C. Voice-Controlled Robots

Google Speech Recognition API is widely used for voice-based robotic control in student projects. A consistent finding is that recognition accuracy drops significantly with background noise and microphone quality. Our experience confirmed this — accuracy dropped from roughly 80% in a quiet room to about 55% in the lab with motors running. This also explains why our attempt to add LLM-based natural language understanding failed: upstream transcription errors made the LLM output unreliable.

D. Obstacle Avoidance with Ultrasonic Sensors

HC-SR04-based obstacle detection is standard in low-cost robotics. Most implementations use a single distance threshold (stop when object detected). ASTRA uses three zones — unrestricted, turning-only, and reverse-only — and applies this logic uniformly across all control modes. This is the key design difference from typical student implementations.

E. Comparison with Existing Approaches

Existing System Feature	Typical Limitation	ASTRA Approach
Single Bluetooth control	~10 m range; drops through walls	UDP over Wi-Fi; network-scale range
No obstacle avoidance	Relies entirely on operator	3-zone firmware safety; buzzer alert; mode-independent
Single control mode	No fallback if interface fails	3 switchable modes from one firmware
Shared motor-control power rail	Brownout resets; sensor errors	Dual buck converter; isolated rails
No timeout safety	Robot runs indefinitely if link drops	5-second inactivity auto-stop
No HRI feedback	Operator has no visual state cues	TFT LCD eye animation via UART

VI. PROPOSED SYSTEM OVERVIEW

ASTRA is organized around three hardware controllers, each responsible for a different aspect of the robot's behavior. The ESP8266 NodeMCU handles everything related to movement: motor control, obstacle sensing, Blynk cloud connection, and UDP command reception. The Arduino Uno handles the TFT display and eye animations. The Raspberry Pi Pico 2W handles live camera streaming independently. A Python application running on a laptop provides voice recognition, gesture detection, and UDP command dispatch.

All three movement control modes — Remote, Voice, Gesture — converge into a single char cmd[] register in the ESP8266 firmware. Whatever sets that register (Blynk callback, UDP packet from voice, or UDP packet from gesture), the same obstacle-checking motor control function executes. This was the key architectural decision that allowed us to have one safety system instead of three.

Fig. 1 would show the overall block diagram of ASTRA's architecture. Fig. 2 would show the power distribution layout. Fig. 3 would show the software module structure on the Python host side.

VII. SYSTEM ARCHITECTURE

Layer	Controller	Responsibilities	Interfaces Used
Primary Control	ESP8266 NodeMCU	Motor PWM, Blynk IoT, UDP server, obstacle sensing, mode FSM, serial TX	Wi-Fi TCP (Blynk), UDP port 4210, UART TX, GPIO PWM
HRI Display	Arduino Uno	TFT LCD eye rendering, blink/speak/sleep animation	UART RX from ESP8266, SPI to TFT
Video Streaming	Raspberry Pi Pico 2W	Wi-Fi live camera feed, independent of motion system	Wi-Fi (separate stream)
Host Processing	Laptop (Python)	Speech recognition, gesture CV, UDP dispatch, TTS	UDP to ESP8266, microphone, webcam

The ESP8266 main loop runs `server_udp()` on every iteration before anything else. This ensures mode-transition commands from the Python host are caught promptly even when the robot is in Remote mode and Blynk is active. The `Blynk.run()` call then services the cloud connection. Depending on the mode variable, the motor control function is called with the current `cmd` value.

As shown in Fig. 1, the ESP8266 simultaneously manages a Blynk TCP connection and a UDP listener on port 4210. These two channels coexist because Blynk's non-blocking `Blynk.run()` and UDP's non-blocking `parsePacket()` can both be polled within a single loop iteration without blocking each other.

VIII. HARDWARE COMPONENTS

Component	Qty	Key Specification	Role
ESP8266 NodeMCU v1	1	Tensilica L106, 80 MHz, 802.11b/g/n, 3.3V logic	Main controller: motors, Wi-Fi, obstacle, serial
Arduino Uno R3	1	ATmega328P, 16 MHz, 5V, hardware UART on pin 0	HRI co-processor, drives TFT display
Raspberry Pi Pico 2W	1	RP2350, 150 MHz dual-core, CYW43439 Wi-Fi	Independent live streaming
Pi Camera Module	1	OV5647, 5MP, CSI-2	Forward camera feed to laptop
L298N Motor Driver	2	Dual H-bridge, 2A/channel [3]	Controls 2 motors each, PWM speed control
DC Geared Yellow Motors	4	6V, hobby gear-reduced DC motors	Wheel drive — all four wheels
HC-SR04 Ultrasonic Sensor	1	40 kHz, 2–400 cm range [4]	Forward proximity measurement
Passive Piezo Buzzer	1	Driven by <code>tone()</code> at 2000 Hz	Obstacle alert

Component	Qty	Key Specification	Role
1.8" TFT LCD (ST7735)	1	128×160 px, SPI	Eye animation display
SG90 Servo Motor	1	9g micro, 0–180°	Camera/sensor directional scanning
Buck Converter — Motors	1	XL4016, 12V → 5V at 3A	Dedicated motor driver logic rail
Buck Converter — Control	1	LM2596, 12V → 5V at 2A	ESP8266, Arduino, sensor rail
12V Battery	1	Main source	Motor power + buck converter input
4-Wheel Dual-Layer Chassis	1	Acrylic frame with motor mounts	Robot body

IX. SOFTWARE TECHNOLOGIES

Module / Library	Platform	Purpose
Embedded C, Arduino IDE	ESP8266 NodeMCU	Motor firmware, Blynk virtual-pin callbacks, UDP server, obstacle logic, serial HRI output
Blynk IoT SDK v1	ESP8266	Mobile remote control; virtual-pin events; terminal widget for distance display
WiFiUDP Library	ESP8266	UDP listener on port 4210 for voice and gesture commands
Python 3.x — main.py	Laptop	Top-level FSM: wake word, mode selection, module invocation
speech.py — SpeechRecognition	Laptop	Google Speech API transcription; pyttsx3 TTS for robot verbal responses
voice_mode.py — threading, socket	Laptop	Timed movement threads, stop_event interrupt, UDP send to ESP8266
gesture.py — OpenCV, MediaPipe	Laptop	Hand landmark detection, finger count, majority-vote smoothing, UDP send
mode_comm.py — socket	Laptop	Utility UDP send function shared by other modules
ai_brain.py — requests (disabled)	Laptop	Ollama LLM API for natural language understanding — disabled (see Section XVI)

X. POWER MANAGEMENT ARCHITECTURE

A. The Problem

In our first working prototype we ran everything off a single battery output: motors, motor drivers, ESP8266, Arduino, and the ultrasonic sensor all shared the same rail. This did not work. When all four motors reversed direction simultaneously during testing, the supply voltage on our multimeter dropped from 7.2V to somewhere below 3V instantaneously. The ESP8266 reset. The sensor started returning 0 cm consistently. Wi-Fi kept dropping after every significant motor event. We spent two days thinking it was a firmware bug before we measured the rail during motor operation.

The root cause is straightforward: DC motors have high inrush current when they start or reverse. Four motors at once can demand over 6 A for a short period, pulling down the supply voltage across the shared

impedance of the wiring and battery internal resistance. The ESP8266 resets at roughly 3.0V — well within the range of these transients.

B. The Fix: Dual Buck-Converter Architecture

We solved this by splitting the power system into two completely separate regulated rails:

- Buck Converter 1 (XL4016, 12V to 5V at 3A): Powers only the L298N motor driver logic pins. This converter absorbs all the dynamic current demand from the motor drivers and is isolated from the control electronics.
- Buck Converter 2 (LM2596, 12V to 5V at 2A): Powers the ESP8266 NodeMCU VIN pin, the Arduino Uno, and the HC-SR04 sensor. This rail stays clean regardless of what the motors are doing.

After this change, we had zero ESP8266 resets, stable sensor readings, and consistent Wi-Fi in all subsequent sessions. The two L298N drivers are split between this architecture — each driver handles two motors, distributing the current load across two independent H-bridge ICs.

Fig. 2 (power distribution diagram) would show the 12V battery splitting into two buck converter paths, with the motor rail and control rail clearly separated.

XI. REMOTE MODE

Remote mode is the default state when the robot powers on. The ESP8266 connects to the Blynk cloud over Wi-Fi, and the operator controls the robot from the Blynk mobile app. The app layout uses a game-controller arrangement: forward/backward on the left side, left/right on the right, and a master power switch at the top. That power switch is implemented as a software interlock — when it is off, no motor commands execute regardless of button presses. This prevents accidental movement during setup.

Three PWM speed sliders (V6, V7, V8) control forward speed, right-turn speed, and left-turn speed independently. We added these because we found that using the same speed for turning as for driving forward made the robot turn too wide in tight spaces. The sliders let us tune this in real time without reflashing the firmware. A Blynk terminal widget on V4 shows the ultrasonic distance reading live, updated only when the value changes to avoid unnecessary API calls.

Obstacle Logic (applies to all modes)

The safety logic is implemented in the `blynk_calling()` function and executes identically for Remote, Voice, and Gesture modes. The three distance zones are:

Distance (d)	Permitted Movements	Buzzer
$d > 30 \text{ cm}$	Forward, Left, Right, Backward — all unrestricted	Off
$20 \text{ cm} < d \leq 30 \text{ cm}$	Left and Right only; forward blocked	On if forward attempted
$0 < d \leq 20 \text{ cm}$	Backward only; turns and forward blocked	On if any blocked command attempted

Mathematically: let D denote the measured forward distance. Movement permission $M(\text{cmd}, D)$ is defined as:

$$M(\text{forward}, D) = 1 \text{ iff } D > 30$$
$$M(\text{turn}, D) = 1 \text{ iff } D > 20$$
$$M(\text{backward}, D) = 1 \text{ for all } D$$

The buzzer activates with `tone(buz, 2000)` when a blocked command is held, and silences with `noTone(buz)` on release. Backward movement has no distance restriction because no rear sensor exists.

XII. VOICE MODE

A. Why We Replaced Bluetooth

Our first attempt at voice mode used an HC-05 Bluetooth module to relay Python-generated strings from the laptop to the ESP8266. Range testing showed reliable operation only within about 8 meters in open space, dropping to 3–4 meters through a wall. More problematically, Bluetooth had to be re-paired after power cycles, and motor EMI caused occasional disconnects. We switched to UDP over Wi-Fi and never went back. UDP is connectionless so there is no pairing state to manage, and the range is limited only by the network coverage.

B. How Voice Mode Works

The `speech.py` module captures audio through the system microphone, adjusts for ambient noise, and sends it to Google Speech Recognition. The recognized text comes back as a string. `voice_mode.py` then checks for keywords like "forward", "left", "stop", and maps them to two-character UDP command codes sent to the ESP8266 on port 4210. The average time from speech to robot response is about 900–1200 ms, almost all of which is the Google API round-trip.

Movement commands are executed in daemon threads using `start_movement()`. A `threading.Event()` (`stop_event`) lets a new command interrupt a running movement before its timer expires. A `threading.Lock()` prevents race conditions when commands arrive in quick succession.

C. Fixed Durations and Timeout Safety

Voice Command	UDP Code	Duration
"forward"	"01"	2.0 s
"backward"	"00"	0.5 s
"left"	"03"	1.0 s
"right"	"02"	1.0 s
"stop"	"04"	Immediate
"gesture mode"	"u"	Mode switch
"remote mode"	"r"	Mode switch
"shutdown"	"s"	Sleep mode

The 5-second inactivity timeout in the ESP8266 firmware automatically stops all motors if no UDP packet arrives within 5 seconds of the last command. This means the robot cannot keep moving indefinitely if

speech recognition stops working or the network drops. The trade-off is that the operator must re-issue a command at least every 5 seconds for sustained movement — we considered this acceptable for safety.

XIII. GESTURE MODE

A. Hand Detection

gesture.py opens the laptop webcam using OpenCV's V4L2 backend, flips each frame horizontally for mirror correction, converts it to RGB, and passes it to MediaPipe Hands initialized with `max_num_hands=2`, and detection/tracking confidence of 0.7. MediaPipe returns 21 3D landmark coordinates per detected hand.

B. Finger Counting and Command Mapping

Finger extension is determined by comparing tip landmark positions against the corresponding second-joint landmarks. The thumb uses a horizontal comparison (`landmark[4].x` vs `landmark[2].x`). All other fingers use a vertical comparison (`tip.y < joint-2.y` in image coordinates means extended). The summed count of extended fingers maps to:

Fingers Extended	UDP Code	Movement
0 (fist)	"00"	Backward
1	"01"	Forward
2	"02"	Right
3	"03"	Left
4	"04"	Stop
Both hands held for 2+ seconds	Exit signal	Leave gesture mode

C. Smoothing and Cooldown

Without smoothing, the raw per-frame finger count oscillates badly due to detection noise. We found during testing that the robot was changing commands roughly 5–10 times per second without any filtering, which made it useless. The fix was a rolling 7-frame majority-vote buffer: a command is only accepted when it appears in more than half the buffer. Additionally, a 3-second cooldown prevents any new command from being processed within 3 seconds of the last one. Before each command transition, a stop signal is always sent first with a 1-second pause — this prevents inertial collisions when switching directions.

The exit condition for gesture mode is two hands detected simultaneously for more than 2 seconds. This was chosen because it is a natural, deliberate gesture that is unlikely to occur accidentally during one-handed operation.

XIV. HUMAN–ROBOT INTERACTION SYSTEM

We added the TFT eye animation system partly for demonstration value and partly because it makes the robot genuinely more readable. When the robot is turning, the eyes shift in that direction. When the robot is

in shutdown, the eyes close. This lets anyone near the robot understand what it is doing without looking at a phone screen.

A. Hardware Setup

An Arduino Uno R3 drives a 1.8-inch ST7735 TFT LCD (128×160 pixels) via SPI. The Arduino is dedicated entirely to this display function. Offloading display rendering to a separate controller ensures that TFT SPI transactions — which can take several milliseconds — do not interfere with the ESP8266's motor control loop.

B. Eye States and Behavior

ESP8266 cmd Value	Serial String Sent	Eye Animation on TFT
"02" (Right)	"right"	Pupils shift right
"03" (Left)	"left"	Pupils shift left
"01", "00", "04", "u"	"on"	Forward-facing with blink cycle
"s" (Shutdown)	"off"	Eyes closed, sleep visualization

The `eyes_calling()` function in the ESP8266 firmware checks the current cmd value and sends the appropriate string over `Serial.println()`. The Arduino reads this continuously on its hardware UART RX pin and updates the display. Response time from command change to eye update is under 200 ms.

XV. SERIAL COMMUNICATION: ESP8266 TO ARDUINO UNO

The UART link runs from ESP8266 GPIO1 (UART0 TX) directly to Arduino Uno pin 0 (hardware UART RX) at 9600 baud, 8N1. No `SoftwareSerial` library is needed on the Arduino because hardware UART on pin 0 is used directly.

Why No Voltage Divider Was Needed

This is worth explaining clearly because it is a common point of confusion. The ESP8266 is a 3.3V device. Its UART TX pin outputs approximately 3.3V for a logic HIGH. The Arduino Uno is a 5V device. Normally, you would expect to need a level shifter or voltage divider to connect a 3.3V output to a 5V input.

However, the ATmega328P datasheet specifies the minimum input HIGH voltage (V_{IH}) as $0.6 \times V_{CC}$. At $V_{CC} = 5V$, this is $0.6 \times 5 = 3.0V$. Since the ESP8266 TX output of 3.3V exceeds this 3.0V threshold, the Arduino reliably interprets it as HIGH. Direct connection is electrically safe and functionally correct.

The reverse direction — Arduino TX (5V HIGH) to ESP8266 RX — is not implemented in ASTRA. If it were, a voltage divider would be mandatory: the ESP8266's GPIO input maximum is 3.6V, and 5V from the Arduino would damage it. Since ASTRA only needs one-way communication (ESP8266 sends state, Arduino receives), this is not a problem for the current design.

XVI. AI INTEGRATION ATTEMPT (OLLAMA)

We had planned to add natural language understanding to voice mode so the robot could respond to semantically equivalent phrases like "go ahead" or "move forward" rather than requiring exact keyword matches. The `ai_brain.py` module was written to call a locally running Ollama server with TinyLlama to classify arbitrary speech input into movement commands.

It did not work in practice, and the reason was upstream: our laptop microphone produced too many transcription errors under lab conditions (motor noise, room reverb). Google Speech often returned garbled or incomplete strings. TinyLlama's responses to those garbled inputs were inconsistent — sometimes it returned a clean command, sometimes conversational text that our parser could not handle. The combined effect was that voice accuracy with the LLM was worse than simple keyword matching.

We kept `ai_brain.py` in the codebase as commented-out code. The plan for future work is to re-enable it with a proper noise-canceling USB microphone and local Whisper transcription, which should produce clean enough input for the LLM to work reliably.

XVII. LIVE VIDEO STREAMING

The Raspberry Pi Pico 2W with Pi Camera module handles live streaming completely independently of the motion control system. Its Wi-Fi stack (CYW43439 chip) is separate from the ESP8266's, so streaming bandwidth does not compete with Blynk TCP traffic or UDP command packets.

We originally considered running the camera stream through the ESP8266 but quickly abandoned that idea. The ESP8266 is already managing Blynk, UDP, obstacle sensing, and motor control simultaneously. Adding high-bandwidth Wi-Fi frame transmission would have introduced timing jitter in the obstacle detection loop, which polls every 80–100 ms for safe navigation. Keeping streaming on a dedicated processor was the right call.

The live stream displays on the laptop at roughly 10–15 fps with about 400–600 ms latency. This is useful for situational awareness but not fast enough to use as a real-time collision avoidance sensor — which is why the hardware ultrasonic sensor remains the only safety-critical input. Separately, a Raspberry Pi Zero 2W running a MobileNet-SSD object detection pipeline has been tested and validated as a companion module. This system detects objects such as persons, cats, and bottles from the camera feed and announces detections verbally via `pyttsx3`, demonstrating that lightweight edge inference is feasible on low-cost Pi hardware. Full integration of this detection module into ASTRA's architecture is planned as part of the future scope described in Section XXV-F.

XVIII. COMMAND ENCODING SYSTEM

All three control sources — Blynk callbacks, voice UDP, gesture UDP — write to the same char `cmd[5]` register in the ESP8266 firmware using `strcpy()`. The motor control function reads from this register. This unification is what allows one safety routine to cover all modes.

Code	Source	Action
"00"	All sources	<code>backward()</code>
"01"	All sources	<code>moveForward()</code> — obstacle-checked

Code	Source	Action
"02"	All sources	moveright() — obstacle-checked
"03"	All sources	moveleft() — obstacle-checked
"04"	All sources	stopMotors()
"r"	Python / Voice	Switch to Remote mode (mode=1)
"u"	Python / Voice	Switch to UDP mode (mode=2)
"s"	Python / Voice	Shutdown (mode=0), stop motors

XIX. SYSTEM WORKFLOW

A. Startup

Power on → both buck converters reach regulated 5V → ESP8266 boots and connects to Wi-Fi → Blynk connects → UDP listener starts on port 4210 → Arduino Uno boots, TFT shows eyes → Raspberry Pi Pico 2W boots, starts streaming → Python main.py launched on laptop → robot announces "ASHA robot online. Say hello."

B. Wake Word and Mode Selection

The main.py outer loop continuously calls listen() waiting for "hello". On detection, the FSM moves to mode-selection state and prompts the user. Saying "voice mode" launches run_voice_mode(); "gesture mode" launches run_gesture(); "shutdown" returns to listening. Within voice mode, saying "gesture mode" directly launches gesture mode without returning to the main menu first.

C. Remote Mode

Default state. Blynk buttons set the cmd register through BLYNK_WRITE callbacks. blynk_calling() runs the motor control and obstacle check. Power switch on Blynk must be ON for any motor commands to execute.

D. Voice Mode

run_voice_mode() sends "u" via UDP to switch ESP8266 to mode=2. Movement commands are dispatched as timed threads. No command for 5 seconds triggers auto-stop. Saying "remote" sends "r" to return to Remote mode.

E. Gesture Mode

run_gesture() opens the webcam, starts MediaPipe, and sends commands via UDP after majority-vote stabilization. Two-hand detection for 2 seconds triggers clean exit back to main.py.

F. Sleep Mode

"Shutdown" in any mode sends "s" via UDP. ESP8266 sets mode=0, stops all motors. Eyes close on TFT. Python returns to listening for "hello". Robot is not powered off — it just stops responding to movement commands until reactivated.

XX. EXPERIMENTAL RESULTS

Test	Observed Outcome	Status
Remote mode: forward / backward / left / right	All movements executed correctly	Pass
Remote mode: obstacle at 15 cm, forward pressed	Motor stopped, buzzer activated	Pass
Remote mode: obstacle at 25 cm, turn pressed	Turn executed normally; forward blocked	Pass
Remote mode: PWM sliders adjusted during movement	Speed changed smoothly in real time	Pass
Voice mode: "move forward" in quiet room	Robot moved 2 s; ~1 s total latency	Pass
Voice mode: 5-second inactivity	Auto-stop triggered correctly	Pass
Voice mode: noisy environment (motors running)	~55% recognition accuracy	Partial
Gesture: 0–4 finger count mapping	All 5 gestures mapped correctly after smoothing	Pass
Gesture: two-hand exit	Mode exited after 2 s of sustained two-hand detection	Pass
Gesture: no smoothing (baseline test)	~30% false command rate	Fail — smoothing required
HRI: eye direction matches robot turn	Left/right eyes synchronized correctly	Pass
HRI: shutdown closes eyes	Eyes closed within 200 ms	Pass
Power: four motors reversing simultaneously	Zero resets after dual buck converter install	Pass
Streaming: laptop display	~12 fps, ~500 ms latency	Pass

XXI. ENGINEERING CHALLENGES FACED

A. Unifying Three Communication APIs in One Firmware

Blynk uses integer virtual-pin callbacks. UDP voice mode uses string commands. These are two fundamentally different mechanisms for telling the motor what to do. Our first attempt used separate code branches for each — essentially three parallel motor control routines that got out of sync with each other. We spent two to three nights restructuring this before arriving at the `char cmd[]` register approach, where any source writes to the same variable and one function reads from it. It took more than 50 compile-flash-test cycles to get this working cleanly.

B. Power Rail Instability

Described in Section X. The key debugging challenge was that the failures were intermittent and looked like software bugs. It was only when we put a multimeter on the supply rail during full motor load that we could see the voltage drop. The lesson here was: always check hardware before blaming firmware.

C. Gesture Detection Noise

Without the majority-vote buffer, the gesture mode was essentially unusable for demonstration. The robot oscillated between commands at frame rate. Adding the 7-frame buffer and 3-second cooldown made it stable, but it also meant the robot takes at least 3 seconds to respond to a gesture change. For a slow-moving robot this is acceptable; for anything requiring fast reactions it would not be.

D. Ollama LLM Integration Failure

Detailed in Section XVI. The core lesson: LLM-based command classification only makes sense when the upstream transcription is reliable. With a poor microphone, the LLM receives garbage and produces garbage. No amount of prompt engineering fixes bad input quality.

E. Raspberry Pi Pico for Computer Vision

We initially planned to run gesture recognition on the Raspberry Pi Pico 2W to avoid needing an external laptop. We tested this and found that MediaPipe inference on the RP2350 processor took over 800 ms per frame — about 30 times slower than the laptop. We moved all CV and speech processing to the laptop immediately after that test.

XXII. PERFORMANCE ANALYSIS

Metric	Value	Notes
Remote mode command latency	200–400 ms	Blynk cloud round-trip dominant
Voice mode end-to-end latency	~900–1200 ms	Google Speech API dominant
UDP transmission latency	~5–15 ms	Local Wi-Fi; not the bottleneck
Voice accuracy (quiet room)	~80%	Acceptable for controlled demo
Voice accuracy (noisy lab)	~55%	Microphone hardware limited
Gesture min. command change time	~3.5 s	Buffer settle time + cooldown
Gesture false activation (with smoothing)	<2 per minute	Stable for demonstration
Obstacle zone enforcement	100% across all modes	No mode-specific gap
HRI eye response time	<200 ms	UART + TFT SPI update
Live stream frame rate	~10–15 fps	Pico 2W processing limit
Live stream latency	~400–600 ms	Wi-Fi buffering; not used for safety
ESP8266 resets after dual buck converters	0	Fully resolved

XXIII. ADVANTAGES

- Three control modes with automatic fallback: if one interface fails, the operator switches modes rather than losing control entirely.

- Obstacle safety is firmware-resident and cannot be bypassed by any control path. All three modes go through the same check.
- Internet-scale communication via Wi-Fi/UDP removes the 10-meter Bluetooth ceiling.
- Dual buck-converter power isolation completely eliminates motor-induced controller instability — the most common hardware failure mode in mobile robots.
- Modular Python architecture: voice engine, gesture engine, and AI brain are independent files that can be replaced or updated without touching other modules.
- Command timeout prevents the robot from running indefinitely if the communication link breaks.
- Distributed hardware design means each controller can be upgraded independently — the Pico 2W can be replaced with a more powerful camera processor without changing the ESP8266 firmware.

XXIV. LIMITATIONS

- Wi-Fi dependency: Remote and voice modes stop working if the network goes down. There is no offline fallback for these modes.
- Laptop dependency: Gesture recognition and voice processing require a connected laptop. The robot is not standalone.
- Microphone quality: Recognition accuracy under lab noise conditions is around 55%, which limits the practicality of voice mode in real-world noisy environments.
- Gesture cooldown: The 3-second cooldown is necessary for stability but makes gesture mode feel sluggish for rapid direction changes.
- No rear obstacle sensor: Backward movement has no safety restriction. The operator must be aware of what is behind the robot.
- Single forward sensor: Lateral obstacles within roughly $\pm 45^\circ$ of the forward axis are not detected.
- Fixed IP dependency: The Python UDP client needs to be updated manually when the ESP8266 gets a new DHCP lease.

XXV. FUTURE SCOPE

A. Onboard Processing with Raspberry Pi 4

The most impactful upgrade would be replacing the external laptop with a Raspberry Pi 4 8GB mounted on the robot. The Pi 4 can run TinyLlama inference, local Whisper speech recognition, and OpenCV gesture detection simultaneously. This would make ASTRA fully standalone. The `ai_brain.py` module is already written and waiting for this hardware upgrade.

B. Natural Language Understanding

With a proper noise-canceling USB microphone and local Whisper transcription, the Ollama LLM integration described in Section XVI becomes viable. The goal is for the robot to understand "come here", "go toward the door", or "stop near the table" without requiring exact command keywords.

C. Multi-Sensor Obstacle Detection

Adding rear and side HC-SR04 sensors would eliminate the current blind spots. A rear sensor in particular would make backward movement safe, completing the safety model.

D. GPS and Live Location Tracking

A NEO-6M GPS module with a web-based location display would enable deployment scenarios where the robot's physical location must be monitored remotely.

E. Signal-Loss Safety Mode

A watchdog routine monitoring Wi-Fi RSSI and UDP packet heartbeat intervals would trigger an autonomous safe-stop sequence when connectivity is lost, preventing runaway behavior in critical deployments.

F. Object Detection

As a parallel development to ASTRA, we have already implemented a real-time object detection system using MobileNet-SSD on a Raspberry Pi Zero 2W. The system detects objects across multiple classes (person, cat, bottle, chair, dining table, and others), applies confidence-based filtering (threshold > 0.5), renders bounding boxes with labels on the video feed, and uses pyttsx3 to verbally announce detected objects in real time via daemon threads. Integrating this directly into ASTRA — mounting the Pi Zero 2W on the robot chassis and connecting its detection output to the ESP8266 — would give the robot genuine environmental awareness. Upgrading to a Raspberry Pi 4 with a YOLO-based model would further extend detection accuracy and class coverage.

XXVI. CONCLUSION

ASTRA works. That sounds like an obvious thing to say, but getting a system this integrated to actually behave reliably took significantly more engineering effort than any of us expected when we started. The firmware unification challenge alone took three nights to resolve. The power instability took two more days to diagnose correctly. The gesture smoothing was a complete redesign after the first approach failed.

What we built is a multimodal robotic platform where three different control interfaces — a mobile app, voice commands, and hand gestures — all feed into a single motor control pipeline with a single set of obstacle safety rules. The distributed hardware design keeps each subsystem independent and maintainable. The dual buck-converter power architecture is something we would apply to any future mobile robot design without question.

The AI integration did not work this time, but the architecture is ready for it. When the hardware catches up — better microphone, onboard Pi 4 — the `ai_brain.py` module is already in the codebase waiting. ASTRA is a reasonable foundation for that next step.

REFERENCES

- [1] Blynk Inc., "Blynk IoT Platform Documentation," [Online]. Available: <https://docs.blynk.io>. [Accessed: 2024].
- [2] Google LLC, "MediaPipe Hands: On-Device Real-Time Hand Tracking," arXiv:2006.10214 [cs.CV], 2020.
- [3] STMicroelectronics, "L298N Dual Full-Bridge Driver Datasheet," Doc. No. DS0026, 2000.

- [4] Elec Freaks, "HC-SR04 Ultrasonic Ranging Module — User Guide," 2013.
- [5] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [6] Espressif Systems, "ESP8266 Technical Reference Manual," v1.7, 2020.
- [7] Microchip Technology, "ATmega328P Datasheet," DS40002061B, 2020.
- [8] Raspberry Pi Foundation, "Raspberry Pi Pico 2W Product Brief," 2024.
- [9] Texas Instruments, "LM2596 SIMPLE SWITCHER Power Converter Datasheet," SNVS124G, 2013.
- [10] Ollama, "Run Large Language Models Locally," [Online]. Available: <https://ollama.ai>. [Accessed: 2024].

ACKNOWLEDGMENT

We thank Rajiv Gandhi University of Knowledge Technologies (RGUKT), RK Valley for providing the lab facilities and academic support that made this project possible. We also thank our faculty mentors in the Departments of ECE and CSE for their feedback and guidance throughout the development process.

APPENDIX A — KEY SOURCE CODE EXCERPTS

A.1 Obstacle Safety Logic — blynk_calling() (Astra.ino)

This function is identical for Remote mode (mode==1) and UDP mode (mode==2). The same distance check applies to both Blynk-sourced and UDP-sourced commands.

```
void blynk_calling() {
  // Only check distance for forward/turn commands
  if (strcmp(cmd, "00") != 0 &&
      (strcmp(cmd, "01") == 0 || strcmp(cmd, "02") == 0 ||
       strcmp(cmd, "03") == 0)) {

    int distance = ultra();
    delay(80);

    if (distance < 30 && distance > 0) {
      // Amber zone: turns allowed, forward blocked
      if (distance > 20 && strcmp(cmd, "02") == 0)
        moveright();
      else if (distance > 20 && strcmp(cmd, "03") == 0)
        moveleft();
      else {
        stopMotors();
        tone(buz, 2000); // Red or amber with forward
      }
    } else if (distance > 30) {
      // Green zone: all forward/turn movements allowed
      noTone(buz);
      if (strcmp(cmd, "01") == 0) moveForward();
      else if (strcmp(cmd, "02") == 0) moveright();
      else if (strcmp(cmd, "03") == 0) moveleft();
    }
  }
}
```

```
    }  
    } else if (strcmp(cmd, "00") == 0) {  
        backward();    // Backward: no distance restriction  
        noTone(buz);  
    } else {  
        stopMotors();  
        noTone(buz);  
    }  
}  
}
```

A.2 Gesture Majority-Vote Smoothing — gesture.py

The buffer holds the last 7 detected commands. A command is dispatched only when it holds a strict majority (>3 out of 7) and the cooldown has expired.

```
command_buffer.append(detected_command)  
if len(command_buffer) > BUFFER_SIZE:  
    command_buffer.pop(0)  
  
most_common = max(set(command_buffer),  
                  key=command_buffer.count)  
  
if command_buffer.count(most_common) > BUFFER_SIZE // 2:  
    if most_common != current_command:  
        if time.time() - last_command_time > COOLDOWN_TIME:  
            # Safety stop before new command  
            if current_command is not None:  
                send_command("00")  
                time.sleep(1)  
            send_command(most_common)  
            current_command = most_common  
            last_command_time = time.time()
```

A.3 Threaded Movement Execution — voice_mode.py

Each voice command runs in a daemon thread. The stop_event allows any subsequent command to interrupt the current movement before its timer expires.

```
def start_movement(command_code, duration):  
    global movement_thread, stop_event  
    with movement_lock:  
        stop_event.set()          # Cancel previous movement  
        send_command("04")        # Explicit stop  
        time.sleep(0.1)  
        stop_event = threading.Event()  
  
def movement_worker():  
    send_command(command_code)  
    start = time.time()  
    while time.time() - start < duration:  
        if stop_event.is_set():  
            return                # Interrupted by new command  
        time.sleep(0.05)  
    send_command("04")            # Natural end of movement
```

```
movement_thread = threading.Thread(  
    target=movement_worker, daemon=True)  
movement_thread.start()
```